

2026 年 5 月 21 日 Version x.x

ハッカソン ～計画書と設計書～

チーム 2

24G1146 : 川又 快斗

24G1053 : 熊本 朱純

24G1133 : 安尾 晃貴

25G1084 : 陳 夏薇

25G1089 : 常世田 隆一

25G1110 : 早川 和輝

25G1115 : 福島 健一

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの成果物に対する有効指標 MOE	1
1.1.2	有効指標 MOE に対応する性能指標 MOP	2
1.1.3	システムテストで評価する性能指標 MOP	2
1.1.4	結合テストで評価する性能指標 MOP	3
1.1.5	単体テストで評価する性能指標 MOP	3
第 2 章	システムの詳細設計	5
2.1	部品一覧	5
2.1.1	音・演出系の部品一覧	5
2.2	楽器制御および音声出力の構成	6
2.2.1	音・演出系のシステム構成	6
2.3	ソフトウェア設計	6
2.3.1	音・演出系のソフトウェア設計	6
2.4	ファイル構成	7
2.4.1	音・演出系のファイル構成	7
2.5	オブジェクトおよび関数設計	8
2.5.1	オブジェクト設計 (クラス図)	8
2.6	処理手順の設計	9
2.6.1	音・演出系の処理手順	9
2.7	テストの設計	11
2.7.1	音テスト設計	11

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの成果物に対する有効指標 MOE

本プロジェクトにおける有効指標（MOE: Measure of Effectiveness）は、ステークホルダやユーザの視点から、開発された成果物が当初の目的をどの程度達成し、期待される価値を提供できているかを評価するための最上位の指標である。これは最終的な受入テストにおける合否判定の基準となり、システムが運用環境においてミッション目標を達成できているかを確認する役割を果たす。全体システムにおける目標のうち、特に音響演出および音楽的表現に特化した有効指標を以下に定義する。

1.1.1.1 音響演出および輪唱表現における有効指標

音響系における第一の有効指標は、指揮者の身体動作に対する音響出力のリアルタイムな追従性と、それに伴う直感的な操作感の獲得である。ユーザによる指揮棒の初回の振り下ろし動作による演奏開始指示や、その後の往復運動における周期の変化に対し、生成される音響出力が人間の知覚可能な遅延や違和感を伴わずに完全同期していることを重視する。専門的な音楽知識や機器操作の習熟を必要とせず、純粋な身体動作のみによって演奏のテンポや開始タイミングを意図通りに制御できているという確信的な操作体験を利用者に提供できているかを評価の主眼とする。

第二の有効指標は、分散配置された複数デバイスの協調動作による、輪唱表現の音楽的な成立と安定性である。本システムの対象楽曲である「かえるの合唱」において、複数の楽器制御ユニットが適切な時間差を保ってシリアル連携を行うことにより、輪唱としてのアンサンブルが音楽的に破綻することなく美しいストリングスとして演奏され続ける必要がある。デバイス間の通信遅延やマイコンの処理負荷の増大といった不安定要因に起因する拍タイミングのズレが、演奏全体の一体感や統一感を損なうことなく、高い信頼性のもとで維持されていることを定量的な成功基準とする。

第三の有効指標は、提供される音響フィードバックがもたらす体験型コンテンツとしての没入感とエンターテインメント性の獲得である。システムから出力されるリアルタイムな音声波形が、利用者を単なる作品の鑑賞者にとどめず、自らがオーケストラを統括しているという深い当事者意識と作品世界への没入感へと誘っているかを重視する。最終的なデモンストレーションにおいて、体験者のみならず周囲の観客にとっても、聴覚的な多角的フィードバックを通じてエンターテインメント性を十分に体感できる演出システムとして総合的に成立していることをプロジェクトの最終的な妥当性確認の目標とする。

1.1.2 有効指標 MOE に対応する性能指標 MOP

本プロジェクトにおける最上位の有効指標（MOE）である「体験型コンテンツとしての没入感の獲得」および「破綻のない輪唱表現の音楽的成立」を技術的に検証するため、各要素成果物が満たすべき技術的要件を客観的測定および主観的評価の双方からアプローチ可能な性能指標（MOP）として定義する。

1.1.2.1 音響演出における性能指標の定義

特に、自動合奏システムが生成する音声出力の品質と再現性を担保するため、客観的評価指標として実音と再現音の周波数スペクトルを比較する対数スペクトル距離（LSD: Log Spectral Distance）を採用する [2]。LSD はシステムが生成する合成波形がどの程度正確に理論上の音響特性を再現できているかをデシベル（dB）単位の物理量として定量化する指標であり、操作感の一体感を裏付ける技術的根拠となる。また、利用者が知覚する操作の直感性や作品世界への参加感を定量化するため、主観的評価指標として平均意見評点（MOS: Mean Opinion Score）を導入する [3]。これにより、エンドツーエンドの遅延時間やジッタといった時間軸上の制御性能だけでなく、最終的なアウトプットである「音」そのものの品質を多角的に評価する網羅的な性能指標を構築する。

1.1.3 システムテストで評価する性能指標 MOP

システムテストフェーズでは、指揮棒型デバイスの動作入力から Processing を介した外部スピーカーからの実音出力にいたる、システム全体を統合した環境における最終的な応答性能および表現品質を評価する。

1.1.3.1 システム全体における音響評価指標と目標値

時間軸における性能指標としては、指揮棒の振り下ろし動作（ダウンビート）を検知してから音声再生および視覚演出が開始されるまでの総合遅延（エンドツーエンド・レイテンシ）を測定し、先行研究の指針に基づく 10ms 以下を目標値として検証を行う。これに加え、システム全体の有効性を総合的に評価する指標として、アンケート調査に基づく主観的評価（MOS）を実施する [3]。このテストでは、システムから提供される視覚的・聴覚的フィードバックが体験者および観客にとって「どれくらい自然に聞こえたか」、および「どの程度の没入感を得られたか」を 5 段階の尺度で評価する。体験型演出システムとしての実用レベルおよびエンターテインメント性を十分に備えていることの成否判定基準として、システムテストにおける総合評価の目標値を MOS 評点 4.0 以上と定義し、その妥当性を確認する。

1.1.4 結合テストで評価する性能指標 MOP

結合テストフェーズでは、センサ制御部、同期制御部、および複数の楽器制御部（Arduino）の間でデータが相互に接続された際、通信の揺らぎや同期ジッタが音声出力の同時性に与える影響を検証する。

1.1.4.1 複数ユニット間における多音同時発音指標

音響系における具体的な性能指標としては、複数台の楽器制御ユニットから同時に発音トリガー（Hz 数値を含むシリアル文字列）が送出された際における、Processing（音声合成ユニット）側での多音同時発音処理の正確性を評価対象とする。シリアル通信網の負荷が増大したマルチクライアント環境下においても、Processing 側のイベントリスナがデータの取りこぼしやバッファオーバーフローを起こすことなく、受信したすべての周波数データをミリ秒単位で正確にパースし、ミキサーオブジェクトを介して各音声波形をリアルタイムにミキシング・合成できるかを確認する。通信の遅延や処理の不整合に起因する各パート間の発音タイミングのズレ（ジッタ）を極小に抑え、輪唱としての「かえるの合唱」がストリングスアンサンブルとして破綻なく成立することを結合テストの合格条件とする。

1.1.5 単体テストで評価する性能指標 MOP

単体テストフェーズでは、個別の機能モジュールが独立して期待通りの性能を発揮しているかを検証する段階であり、音響系においては PC 上で動作する音声合成ユ

ニット単体での音声波形生成能力の妥当性を評価する.

1.1.5.1 音声合成モジュールにおける波形再現性指標

具体的な性能指標として, あらかじめ楽譜データ (MusicData) として定義された理論値としての周波数スペクトル (本物のスペクトル: $S_1(k)$) と, Processing 内部の発振器 (オシレータ) から実際に生成されてサンプリングされた音声波形の周波数スペクトル (再現音のスペクトル: $S_2(k)$) との誤差を, 対数スペクトル距離 (LSD) を用いて算出する [2]. 評価式としては, 全周波数ビン数 N における両スペクトルの対数比の二乗平均平方根を適用し,

$$LSD = \sqrt{\frac{1}{N} \sum_{k=1}^N \left(20 \log_{10} \frac{S_1(k)}{S_2(k)} \right)^2} \quad (1.1)$$

として計算される物理的な誤差量を測定する. 単体テストにおける音声合成の合格基準として, 実用レベルに達していることを示す 1~2 dB の誤差範囲 (LSD 値) に収まることを目標値として定義し, オシレータのパラメータ適用論理やエンベロープ制御プログラムの正確性を定量的に確認する.

第 2 章 システムの詳細設計

本章では、前章で定義した基本設計に基づき、自動合奏システムを実際に構築するための詳細な設計について述べる。本システムは合計 6 台のマイクロコントローラを用いた分散制御構成を採用している。製品分解構造（PBS: Product Breakdown Structure）に基づく部品の選定、ハードウェア構成、各ユニットのソフトウェア設計、および処理手順を明らかにする。

2.1 部品一覧

2.1.1 音・演出系の部品一覧

本システムの発音機能（楽譜再生・音声合成）および物理・視覚演出機能を構成する主要なハードウェア要素について定義する。他のモジュール（感知・同期系）とのインターフェース接続を考慮し、音・演出系に特化した選定を行っている。

本システムにおける主要な制御中枢として、3 台の Arduino UNO R4 WiFi を楽器制御（リズム用）の従属ユニットとして配備する。これらは「楽譜の作成」タスクで定義されたメロディデータをそれぞれ保持し、四分音符 1 拍の長さを基準に正確な発音タイミングを制御するとともに、後述する LED マトリクス表示およびサーボモータの物理駆動を直接制御する役割を持つ。また、楽曲全体のテンポと拍の基礎を維持するため、1 台の Arduino UNO R4 WiFi を楽器制御（ビート用）として専任で配備し、打楽器的なベースリズムのタイミング制御を独立して実行させる。

物理演出のアクチュエータとしては、マイクロサーボモータ（SG92R）を合計 6 個実装する。これは「サーボモータの実装」タスクにおける中核部品であり、「かえるの合唱」を構成する 6 つの音階（ド・レ・ミ・ファ・ソ・ラ）に対して 1 対 1 で対応するよう物理配置される。各音階が発音された瞬間に対応するサーボモータが作動し、紙のカエルのオブジェクトを起き上がらせる動的な視覚演出を具現化する。

2.2 楽器制御および音声出力の構成

2.2.1 音・演出系のシステム構成

本システムにおける音響および演出処理は、独立したタイミング管理を行う合計 4 台の楽器制御用 Arduino と、実際の音声波形生成を担う PC 上の Processing 環境との緊密な連携によって実現される。

まず、Arduino と Processing の連携を担保するシリアル通信網において、各楽器制御用 Arduino は割り当てられたパートの楽譜データを常時スキャンする。内部処理が発音時刻に到達すると、「楽譜から音の数値 (Hz) の出力の作成」タスクに基づき、対象の音階に対応する周波数 (Hz) の数値をシリアル通信経由で PC へと即座に送信する。

PC 側で稼働する音声合成ユニットとしての Processing は、シリアルポートから受信した周波数データを瞬時に解析する「Processing の音出力作成」タスクを実行する。Processing 内のサウンドライブラリに組み込まれた発振器（オシレータ）に対してこの周波数数値を動的に適用することにより、低遅延かつ滑らかな音声出力をリアルタイムに生成し、外部スピーカーから実音を出力させる構成をとる。

さらに、これらの発音処理と完全に同期したマルチモーダルな演出機構として、LED マトリクス表示と物理サーボの同期演出を展開する。具体的には、Arduino UNO R4 WiFi に標準搭載されている 12×8 LED マトリクスを活用し、「カエルの口が開く」といった動的なドット絵アニメーションを表示する「LED マトリクス表示」タスクを実行する。これと同時に、先述した SG92R マイクロサーボモータへとパルス信号を送り、紙のカエルを物理的に跳ね上げさせる「サーボモータの実装」タスクを並行して駆動させ、聴覚と視覚が高度に融合したアンサンブル空間を創出する。

2.3 ソフトウェア設計

2.3.1 音・演出系のソフトウェア設計

音響・演出制御を司るソフトウェア群は、ミリ秒単位の周期処理の中で「楽譜データのシーク」「Hz 出力」「外部機器トリガー」を並行して行う Arduino 側と、「シリア

ル割り込み」「波形合成」を担う Processing 側の協調論理として設計する。

2.3.1.1 楽器制御ユニット (Arduino) の設計

Arduino 側のソフトウェアは、同期制御ユニットからの命令をポーリング受信する通信処理と、自律的な楽譜再生処理の 2 つのタスクを並行実行する。「テンポを変更させる処理の追加」タスクに対応するため、タイマーカウンタによる固定待機ではなく、受信した BPM 値 (拍歩進速度) から 1 拍あたりの実時間を動的に逆算 ($T = 60000/\text{BPM}$ [ms]) する数値モデルを実装する。これにより、演奏の途中でであっても音価の比率を保ったまま、滑らかに再生速度を変更させる処理を実現する。

2.3.1.2 音声合成ユニット (Processing) の設計

Processing 側のソフトウェアは、複数台の Arduino から同時にシリアルデータが流入するマルチクライアント環境を前提として設計する。シリアルポートのイベントリスナ (割り込み処理) により、受信した文字列から正確な周波数 (Hz) を抽出し、ミキサオブジェクトを介して複数の発振器 (オシレータ) を同時に駆動・合成することで、輪唱 (ストリングスアンサンブル) に耐えうる多音同時発音出力を実現する。

2.4 ファイル構成

2.4.1 音・演出系のファイル構成

本モジュールを構成する実装ソースファイル群の責務と依存関係について説明する。ソースコード群は主として、Arduino 側で動作する楽器・演出制御プロジェクトと、PC 側で動作する音声合成プロジェクトの 2 つに大別される。

楽器・演出制御プロジェクト (Arduino 側) においては、4 つのファイルがそれぞれの責務を分担する。メイン処理を担う `InstrumentMain.ino` はプログラムのメインループを包含し、同期信号の解析およびテンポ変更に基づく時間歩進の統括管理を行う。`MusicData.h` は「楽譜の作成」タスクの中核であり、音階の周波数配列や音価配列、および LED マトリクス表示用のビットマップ図形データを構造体配列として定義する。再生エンジンとしての実処理は `InstrumentController.h / cpp` に実装され、楽譜データから音の数値 (Hz) を抽出してシリアルポートへ出力する制御論理を担う。また、`FrogServoController.h / cpp` は「サーボモータの実装」に特化してお

り、可動演出のための角度計算、PWM 信号の生成、およびオブジェクトの起き上がり・戻り動作に伴う遅延制御を実装する。

一方、音声合成プロジェクト（Processing 側）は 3 つのコンポーネントから構成される。メインスケッチである `SoundSynthesizer.pde` は、音声出力環境（オーディオコンテキスト）の初期化とミキシング処理の全体統括を行う。`SerialReceiver.pde` は「Arduino と Processing の連携」を司り、各シリアルポートからの割り込み文字列をバッファリングして正確にパースする処理を実行する。最後に `AudioPlayer.pde` は「Processing の音出力作成」の実装ファイルとして機能し、内部オシレータ（波形発振器）の動的割り当てと、自然な音量変化を生み出すエンベロープ制御を担う。

2.5 オブジェクトおよび関数設計

2.5.1 オブジェクト設計 (クラス図)

本システム全体のクラス構成およびオブジェクト間の相互依存関係を図 2.1 に示す。本システムは、センサー感知から同期制御、各楽器での再生および外部音声合成環境（Processing）にいたるまで、役割ごとにクラスを明確に分離したオブジェクト指向設計を行っている。

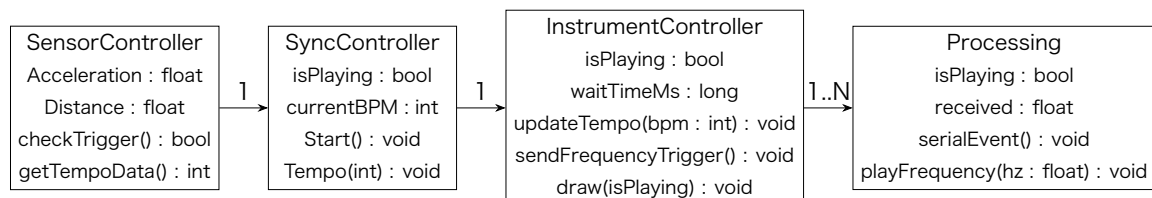


図 2.1 システムのクラス図

また、音響・演出系モジュールを中心とした主要なクラス、構造体、および各コンポーネントの具体的な属性とメソッドの仕様を表 2.1 に定義する。

表 2.1 に示した各クラスとメソッドは、システム要件として掲げられた開発タスクと密接に対応している。まず「楽譜の作成」タスクは、構造体である `MusicData` 内に周波数（Hz）と音価の配列として静的に保持される。このデータに基づき、`InstrumentController` クラスの `sendFrequencyTrigger()` メソッドが呼び出されることで「楽譜から音の数値（Hz）の出力の作成」が実行される。演奏中にテンポの動

表 2.1 クラス設計

クラス名	主な属性	主なメソッド
MusicData(構造体)	MELODY_HZ, MELODY_DURATIONS	(楽譜データの保持)
InstrumentCon- troller	bpm, nextTick, playState	updateTempo(), sendFre- quencyTrigger(), drawFrog- Matrix()
FrogServoCon- troller	currentAngle, noteIndex	moveFrog()
SerialReceiver	receiveBuffer	serialEvent()
AudioPlayer	oscillator, frequency	playFrequency()

的変更を可能にする「テンポを変更させる処理の追加」タスクについては、同クラスの `updateTempo()` メソッドが BPM 情報から次の発音待機時間を再計算することで実現される。また、視覚演出に関連する「LED マトリクス表示」タスクは `drawFrogMatrix()` メソッドが LED マトリクス上のアニメーションパターンを切り替えることで制御され、「サーボモータの実装」タスクは `FrogServoController` クラスの `moveFrog()` メソッドが割り当てられた音階番号に応じて該当のサーボモータを物理駆動することで達成される。さらに、ハードウェアとソフトウェアを仲介する「Arduino と Processing の連携」タスクは `SerialReceiver` クラスの `serialEvent()` がシリアルバッファから Hz 数値を抽出することで担われ、最終的な「Processing の音出力作成」タスクは `AudioPlayer` クラスの `playFrequency()` メソッドが内部の発振器へ周波数を動的適用して音声波形を出力することで完結する。

2.6 処理手順の設計

2.6.1 音・演出系の処理手順

本システムにおける「楽譜作成」から「音の動作確認」に至る一連のランタイム処理の手順について、時系列に沿って解説する。また、システム駆動時における処理の分岐およびデータの流れを図 2.2 のフローチャートに示す。

初期化フェーズにおいて、楽器制御用 Arduino は `MusicData` からあらかじめ「楽譜の作成」タスクによって組み込まれた曲データをロードし、シーク位置を先頭に合わせた待機状態で実行を開始する。同時に、PC 側で待機する Processing は対象と

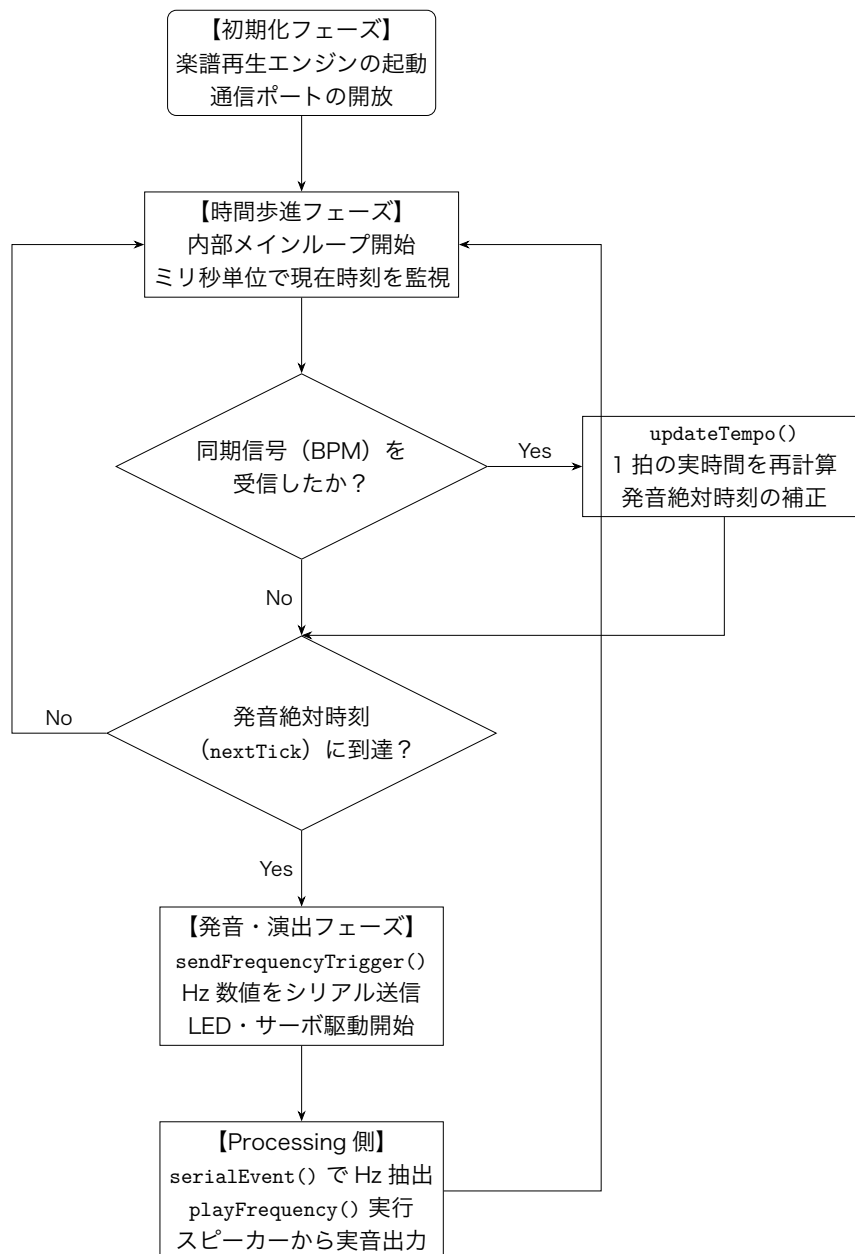


図 2.2 音・演出系モジュールのランタイム処理フローチャート

なるすべてのシリアル通信ポートを開放し、「Processing の音出力作成」タスクを実行するために必要となる波形発振器（オシレータ）やエンベロープオブジェクトのメモリ割り当てを完了させる。

演奏開始後の時間歩進フェーズでは、Arduino が内部のメインループ内でミリ秒単位の現在時刻を常に監視する。この過程で同期制御ユニットから新たなテンポ情報を受信した場合、直ちに `updateTempo` 処理を介入させ、「テンポを変更させる処理の追加」タスクを実行する。これにより、現在の BPM 値から 1 拍の実時間を再計算し、次の発音絶対時刻である `nextTick` を動的に補正する。

算出された発音絶対時刻に到達すると、Arduino 側で `sendFrequencyTrigger` が呼び出され、「楽譜から音の数値 (Hz) の出力の作成」タスクが処理される。これと完全に同一の時間軸において、内部の演出トリガーが連動し、`drawFrogMatrix` による「LED マトリクス表示」タスクと `moveFrog` による「サーボモータの実装」タスクが並行して実行され、ハードウェア側の視覚・物理演出が即座に開始される。

最終的な音声出力フェーズにおいて、Arduino のシリアルポートから送出された周波数データは、「Arduino と Processing の連携」を支える有線シリアル通信網を経由して PC へ高速伝送される。Processing 側のイベントリスナである `serialEvent` がこのデータ流入を検知してパースを行い、抽出された正確な Hz 数値を `playFrequency` の入力引数へと渡す。オシレータがこの周波数に基づいて即座に音声波形を更新し、PC に接続された外部スピーカーから空気振動として実音が出力されることで、一連の自動演奏シークエンスが完結する。

2.7 テストの設計

2.7.1 音テスト設計

本章で構築した音響・演出制御モジュールが、設計段階で規定された各開発タスクの要求仕様を完全に満たしているかを実証するため、学術的かつ多角的な検証アプローチに基づく複合的なテストを計画する。本設計においては、「音の動作確認」の達成度およびテンポ急変時における「動的追従性」の定量的評価を主眼に据える。

第一に、楽譜のデータ整合性および数値 Hz 出力テストを実施する。ここでは `MusicData` 内に格納された「楽譜の作成」タスクのデータ構造が、意図通りの音階および音価として正確にプログラムへロードされているかを検証する。具体的には、

Arduino 単体を PC のシリアルモニタに接続した状態でエミュレート走行を行い、音符のインデックスが切り替わるタイミングごとに、設計値通りの正確な音の数値 (Hz) がシリアルバッファへ遅延なく出力されているかをログ解析によって確認する。

第二に、Arduino と Processing の連携および音の動作確認テストを展開する。Arduino の送出した Hz 文字列が、有線シリアル通信網を介して Processing 側に到達する過程を監視し、文字化けやデータの packets 欠落に起因するパースエラーが発生していないかを実機検証する。複数台の楽器制御ユニットから同時にデータが送信されたマルチクライアント環境下においても、Processing がこれらを低遅延でミキシングし、スピーカーからクリアな「かえるの合唱」の輪唱として聴取可能であるかを調べることで、システム全体の統合的な音の動作確認とする。

第三に、動的テンポ追従および処理追加テストを行う。演奏の途中で、同期制御ユニット側から BPM の変更指示 (例えば BPM=60 から BPM=120 への即時変更など) を意図的に投入する動的負荷試験を行う。この際、生成される音声のピッチ (周波数 Hz) に不要なゆらぎや歪みを生じさせることなく、発音の間隔 (四分音符の物理的な実時間) だけが正確かつ滑らかに変速され、「テンポを変更させる処理の追加」タスクが設計通り機能しているかを測定する。

第四に、LED マトリクスおよびサーボモータ演出完全同期テストを実行する。Processing のオシレータから音声波形が物理出力される瞬間を基準とし、対象となる Arduino 側の「LED マトリクス表示」の絵柄切り替え動作、および SG92R マイクロサーボモータによるカエルオブジェクトの起き上がり動作 (「サーボモータの実装」確認) の 3 点間における時間的なズレを測定する。高速度カメラを用いたフレーム解析等により、人間が知覚可能なタイムラグが発生せず、完全なマルチモーダル同期演奏が達成されているかを定量的に検証する。

参考文献

- [1] David Wessel and Matthew Wright, “Problems and Prospects for Intimate Musical Control of Computers”, Computer Music Journal, Vol.26, No.3, pp.11–22, 2002.
- [2] Yhonatan Gayer, “SHroom: A Python Framework for Ambisonics Room Acoustics Simulation and Binaural Rendering”, arXiv preprint

arXiv:2603.27342, 2026.

- [3] ITU-T Study Group 12, “Mean Opinion Score (MOS) terminology”, ITU-T Recommendation P.800.1, Jul. 2016.